
Transaktionen

BITLC | Tom Selig
11. August 2026

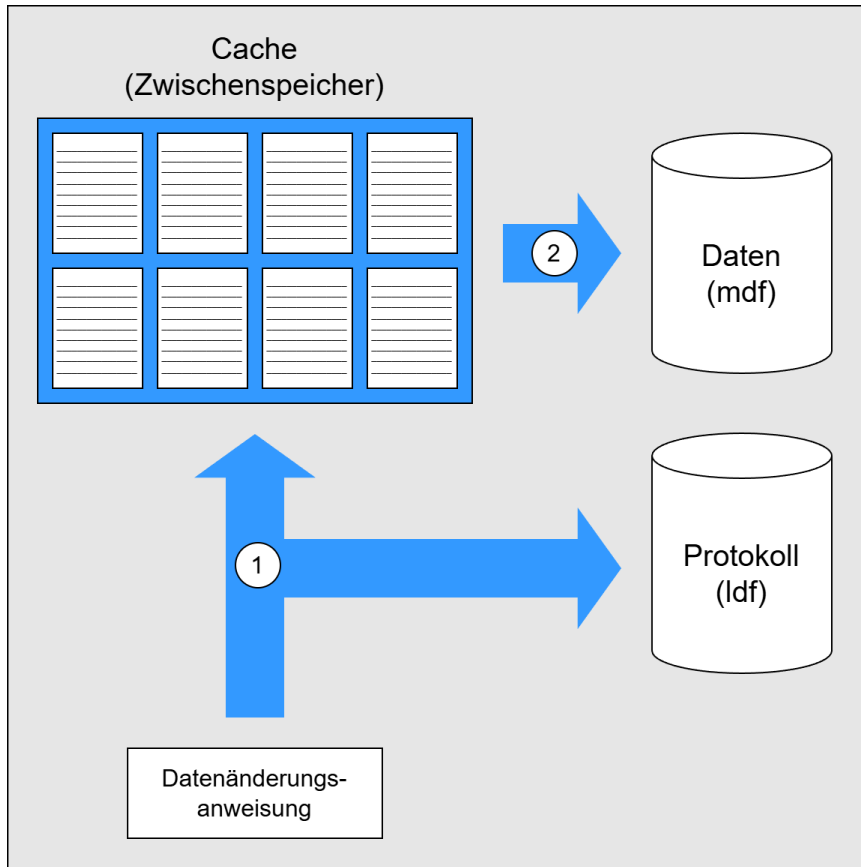
Grundlagen

- Transaktionen sind Arbeitseinheiten, die aus mehreren einzelnen Anweisungen bestehen können.
- Ein Transaktion wird entweder komplett oder gar nicht ausgeführt. Sie kann aber niemals nur teilweise ausgeführt werden.
- Transaktionen sollen die Integrität der Daten in Mehrbenutzer-System und bei Fehlern sicherstellen.
- Der SQL-Server behandelt intern alle Anweisungen der DML und der DDL als Transaktionen.
- Auch als Benutzer kann man mehrere Anweisungen zu einer expliziten Transaktion zusammenfassen.

ACID-Prinzipien

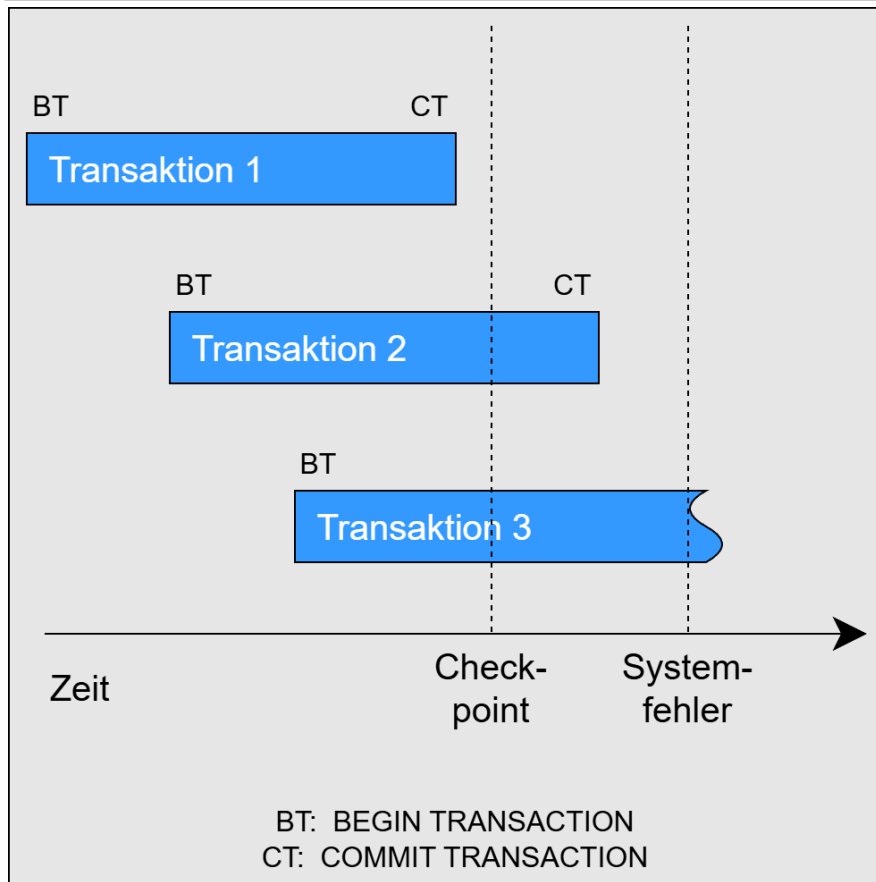
A tomicity (Atomarität)	Die Anweisungen einer Transaktion werden entweder komplett oder gar nicht ausgeführt. Eine Transaktion kann also niemals nur zum Teil ausgeführt werden.
C onsistency (Konsistenz)	Die Datenbank wird ausgehend von einem konsistenten Zustand durch eine Transaktion in einen weiterhin konsistenten Zustand überführt. Dies muss aber nicht zwingend eine neuer/anderer Zustand sein.
I solation	Während eine Transaktion läuft kann die Konsistenz der Datenbank nicht garantiert werden. Eine Transaktion sollte daher nicht auf Daten zugreifen, die in einer anderen parallel laufenden Transaktion bearbeitet werden. Dies kann über die Isolationsstufe reguliert werden.
D urability (Dauerhaftigkeit)	Nach einer gültig abgeschlossenen Transaktion müssen die Daten in jedem Fall dauerhaft in der Datenbank gespeichert werden. Auch wenn nach der Ausführung ein Systemfehler das Schreiben in die Datendatei verhindert hat (=> Transaktionsprotokoll)

Transaktionsverarbeitung



- Die Änderungen werden zunächst nur im Cache vorgenommen. Dabei wird parallel ein Protokoll geführt.
- Ist die Transaktion erfolgreich, wird das im Protokoll vermerkt.
- Bei einem Abbruch werden die Änderungen aus dem Protokoll im Cache rückgängig gemacht.
- In unregelmäßigen Abständen (Checkpoint) werden die Daten des Cache in den Datenbestand übernommen.

Verhalten bei Systemfehlern



- Transaktion 1 war vor dem Checkpoint erfolgreich beendet
=> *keine Aktion notwendig*
- Transaktion 2 war vor dem Fehler beendet, aber nach dem Checkpoint
=> *ausstehende Änderungen werden übernommen (Rollforward)*
- Transaktion 3 war beim Fehler noch nicht abgeschlossen und lief bereits während des Checkpoints
=> *Änderungen bis zum Checkpoint werden rückgängig gemacht (Rollback)*

Explizite Transaktionen

- Explizite Transaktionen sind diejenigen, deren Beginn und Ende im Programmcode angegeben werden.
- Folgende Befehle stehen zur Verfügung:
 - BEGIN TRANSACTION** [*<name>*]
Markiert den Beginn einer expliziten Transaktion.
 - COMMIT** [**TRANSACTION**] [*<name>*]
Bestätigt eine erfolgreiche Transaktion und speichert die Daten.
 - ROLLBACK** [**TRANSACTION**] [*<name>*]
Führt ein Rollback zum Anfang durch und verwirft alle Änderungen.
- Da die Transaktionen nicht in der Lage sind, Fehler selbständig zu erkennen, muss eine Fehlerbehandlung implementiert werden.

Beispiel

```
USE tempdb;
GO
CREATE TABLE TransTest (wert tinyint);      -- Tabelle anlegen
GO

BEGIN TRANSACTION;                           -- Beginn der Transaktion
BEGIN TRY                                    -- Beginn des TRY-Blocks
    INSERT INTO TransTest VALUES (1);
    INSERT INTO TransTest VALUES (2);
    INSERT INTO TransTest VALUES ('a');      -- Erzeugt einen Fehler
    PRINT 'Transaktion erfolgreich';
    COMMIT TRANSACTION;                      -- Bestätigen der Transaktion
END TRY
BEGIN CATCH                                  -- Beginn des CATCH-Blocks
    PRINT 'Nicht erfolgreich';
    ROLLBACK TRANSACTION;                    -- Transaktion zurückdrehen
END CATCH

SELECT wert FROM TransTest;                  -- Ergebnis ansehen
```

Transaktionsebenen

- Mit der System-Variablen @@TRANCOUNT kann die Anzahl offener Transaktionen (die Transaktionsebene) abgefragt werden:

BEGIN TRANSACTION erhöht @@TRANCOUNT um 1.

COMMIT TRANSACTION reduziert @@TRANCOUNT um 1.

ROLLBACK TRANSACTION setzt @@TRANCOUNT auf 0.

```
PRINT @@TRANCOUNT;      -- 0
BEGIN TRAN
    PRINT @@TRANCOUNT;   -- 1
    BEGIN TRAN
        PRINT @@TRANCOUNT; -- 2
    COMMIT
    PRINT @@TRANCOUNT;   -- 1
COMMIT
PRINT @@TRANCOUNT;      -- 0
```

```
PRINT @@TRANCOUNT;      -- 0
BEGIN TRAN
    PRINT @@TRANCOUNT;   -- 1
    BEGIN TRAN
        PRINT @@TRANCOUNT; -- 2
    -- ROLLBACK setzt alle offenen
    -- Transaktionen zurück
ROLLBACK
PRINT @@TRANCOUNT;      -- 0
```


Transaktionsstatus

- Über die System-Funktion **XACT_STATE()** lässt sich der Transaktionsstatus ermitteln:
 - 0:** Es liegt keine aktive Transaktion vor.
 - 1:** Es gibt eine aktive Transaktion, für die noch ein **COMMIT** erfolgen könnte.
 - 1:** Es gibt eine aktive Transaktion, für die aufgrund eines Fehlers kein **COMMIT** mehr erfolgen kann.

```
USE AdventureWorks2019;
GO
--SET XACT_ABORT ON|OFF;
BEGIN TRY
    BEGIN TRANSACTION;
        DELETE FROM Production.Product
        WHERE ProductID = 980;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF (XACT_STATE()) = -1
    BEGIN
        PRINT 'Transaction is uncommittable.'
        ROLLBACK TRANSACTION;
    END;
    IF (XACT_STATE()) = 1
    BEGIN
        PRINT 'Transaction is committable.'
        COMMIT TRANSACTION;
    END;
END CATCH;
GO
```

Risiken bei parallelen Transaktionen

Risiko	Verhalten
Dirty Reads	Ein Schreib-Lese-Konflikt tritt auf, wenn von zwei gleichzeitig ablaufenden Transaktionen die eine Transaktion Daten liest, die von der anderen Transaktion geschrieben bzw. geändert werden, jedoch noch nicht bestätigt (committed) sind.
Lost Updates	Verlorene Updates entstehen, wenn zwei Transaktionen parallel denselben Datensatz modifizieren und nach Ablauf dieser beiden Transaktionen nur die Änderung von einer von ihnen übernommen wird.
Non-Repeatable Reads	Nicht wiederholbares Lesen bezeichnet ein Problem, das auftritt, wenn innerhalb einer Transaktion die gleiche Leseoperation nacheinander inhaltlich unterschiedliche Ergebnisse liefert, weil eine andere Transaktion die Datensätze zwischenzeitlich verändert hat.
Phantom Reads	Suchkriterien treffen während einer Transaktion auf eine unterschiedliche Anzahl an Datensätzen zu, weil eine andere Transaktion zwischenzeitlich Datensätze hinzugefügt, geändert oder entfernt hat.

Dirty Reads

Transaction 1

```
BEGIN TRANSACTION;  
SELECT age FROM users WHERE id = 1;  
-- retrieves 20
```

```
SELECT age FROM users WHERE id = 1;  
-- retrieves 21  
-- (depending on isolation level)  
COMMIT TRANSACTION;
```

Transaction 2

```
BEGIN TRANSACTION;  
UPDATE users SET age = 21 WHERE id = 1;  
-- no commit here
```

```
ROLLBACK TRANSACTION;
```

Lost Updates

Transaction 1

```
BEGIN TRANSACTION;  
DECLARE @cnt MONEY;  
SELECT @cnt = counter  
FROM data WHERE id = 1;  
-- retrieves 100  
  
UPDATE data SET counter = @cnt + 100  
WHERE id = 1;  
COMMIT TRANSACTION;  
-- sets counter to 200
```

Transaction 2

```
BEGIN TRANSACTION;  
DECLARE @cnt MONEY;  
SELECT @cnt = counter  
FROM data WHERE id = 1;  
-- retrieves 100  
  
UPDATE data SET counter = @cnt + 100  
WHERE id = 1;  
COMMIT TRANSACTION;  
-- sets counter to 200
```

Non-Repeatable Reads

Transaction 1

```
BEGIN TRANSACTION;  
SELECT age FROM users WHERE id = 1;  
-- retrieves 20
```

```
SELECT age FROM users WHERE id = 1;  
-- retrieves 21  
-- (depending on isolation state)  
COMMIT TRANSACTION;
```

Transaction 2

```
BEGIN TRANSACTION;  
UPDATE users SET age = 21 WHERE id = 1;  
COMMIT TRANSACTION;
```

Phantom Reads

Transaction 1

```
BEGIN TRANSACTION;  
SELECT name FROM users WHERE age > 17;  
-- retrieves Alice and Bob
```

```
SELECT name FROM users WHERE age > 17;  
-- retrieves Alice, Bob and Carl  
-- (depending on isolation state)  
COMMIT TRANSACTION;
```

Transaction 2

```
BEGIN TRANSACTION;  
INSERT INTO users VALUES (3, 'Carl', 26);  
COMMIT TRANSACTION;
```

Isolationsstufen

Isolationsstufe	Beschreibung	Stärke
READ UNCOMMITTED	Gibt an, dass Zeilen von einer Transaktion gelesen werden können, die von anderen Transaktionen geändert wurden, obwohl dort noch kein COMMIT ausgeführt wurde.	schwach
READ COMMITTED	Gibt an, dass Zeilen von einer Transaktion nicht gelesen werden können, die von anderen Transaktionen geändert wurden, solange dort noch kein COMMIT ausgeführt wurde. Dadurch werden Dirty Reads verhindert.	stärker
REPEATABLE READ	Gibt an, dass eine Transaktion keine Daten lesen kann, die von anderen Transaktionen geändert wurden, für die jedoch noch kein COMMIT ausgeführt wurde. Darüber hinaus können von einer Transaktion gelesene Daten erst nach Abschluss der Transaktion von anderen Transaktionen geändert werden. Dadurch werden zusätzlich Lost Updates und Non-Repeatable Reads verhindert.	noch stärker

Isolationsstufen

Isolationsstufe	Beschreibung	Stärke
SNAPSHOT	Gibt an, dass alle von einer Transaktion gelesenen Daten immer dem Stand entsprechen, der zu Beginn der Transaktion vorhanden war. Auch nacheinander ausgeführte Abfragen auf verschiedene Datenmengen sind immer konsistent zueinander.	sehr stark
SERIALIZABLE	Verwendet physische Sperren, die bis zum Ende der Transaktion gehalten werden. • Die aktuelle Transaktion kann keine Daten lesen, die von anderen Transaktionen geändert wurden, für die aber noch kein COMMIT ausgeführt wurde. • Andere Transaktionen können Daten, die von der aktuellen Transaktion gelesen werden, erst dann ändern, wenn die aktuelle Transaktion abgeschlossen ist. • Andere Transaktionen können erst nach Abschluss der aktuellen Transaktion neue Zeilen mit Schlüsselwerten einfügen, die in den Schlüsselbereich der aktuellen Transaktion fallen.	am stärksten

Isolationsstufen und Risiken

Isolationsstufe	Dirty Reads	Lost Updates	Non Repeatable Reads	Phantom Reads
Read Uncommitted	möglich	möglich	möglich	möglich
Read Committed	unmöglich	möglich	möglich	möglich
Repeatable Read	unmöglich	unmöglich	unmöglich	möglich
Snapshot	unmöglich	unmöglich	unmöglich	unmöglich
Serializable	unmöglich	unmöglich	unmöglich	unmöglich

Sperren

- Der SQL-Server arbeitet mit verschiedenen Arten von Sperren, um die Isolationsstufen für Transaktionen umzusetzen:

Shared Lock

Wird beim Lesen verwendet. Andere lesende Zugriffe sind weiterhin möglich, schreibende Zugriffe werden blockiert.

Update Lock

Ankündigung eines schreibenden Zugriffs. Wird in einen Exclusive Lock umgewandelt, sobald alle anderen Sperren wieder freigegeben sind.

Exclusive Lock

Wird beim Schreiben verwendet. Alle anderen lesenden und schreibenden Zugriffe werden blockiert.

Sperren

- Sperren können auf verschiedenen Ebenen eingerichtet werden:
Datensatz, Seite (8 KB), Block/Extent (8 Seiten), Tabelle oder Datenbank
- Die Aufteilung in versch. Sperrebenen nennt sich Sperrgranularität.
- Der SQL Server arbeitet mit Lock Escalation:

Sobald eine kritische Anzahl an Elementen einer Ebene gesperrt sind, wird die nächst höhere Ebene gesperrt.

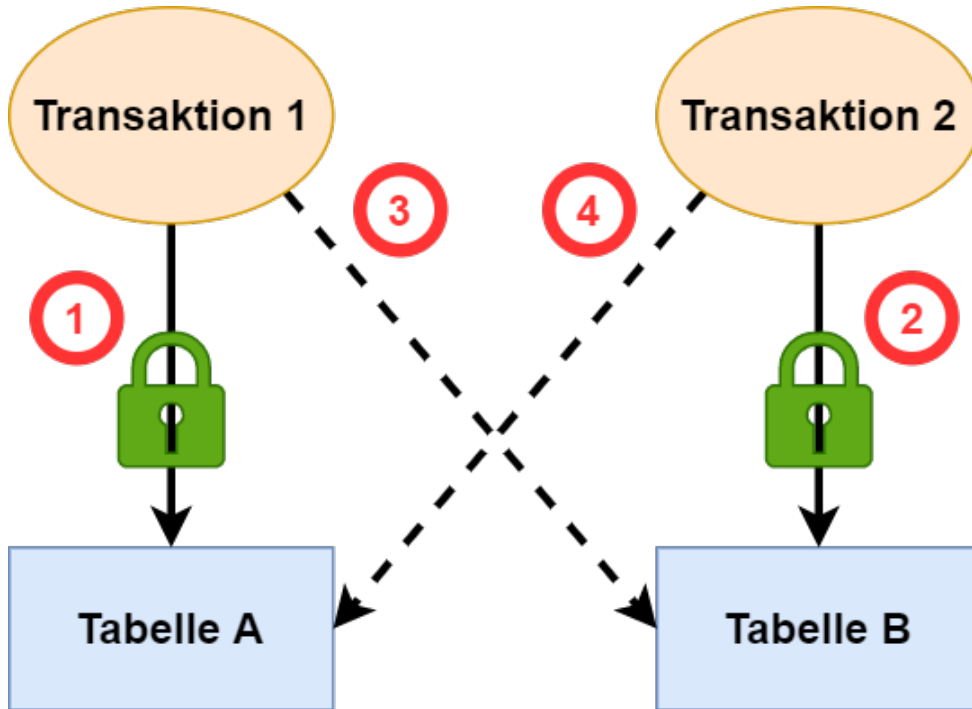
Beispiel:

Wenn viele Datensätze einer Seite gesperrt sind, wird stattdessen die gesamte Seite gesperrt. Sind wiederum viele Seiten gesperrt, werden entweder die entsprechenden Blöcke oder die ganze Tabelle gesperrt.

Deadlocks

- Sogenannte Deadlocks treten auf, wenn zwei Transaktionen jeweils eine Ressource mit Sperren belegt haben, auf die die andere Transaktion zugreifen möchte.
- Der SQL-Server erkennt Deadlocks automatisch und bricht eine der beiden Transaktionen ab (das Deadlock-Victim) und führt für diese ein **ROLLBACK** durch.
- Es wird die Transaktion abgebrochen deren Abbruch weniger „Kosten“ verursacht, oft ist das die später gestartete Transaktion.
- Die abgebrochene Transaktion muss manuell oder durch die Anwendung neu gestartet werden (Fehlercode 1205 abfangen).

Deadlocks



- 1) Transaktion 1 sperrt Tabelle A.
- 2) Transaktion 2 sperrt Tabelle B.
- 3) Transaktion 1 will auf Tabelle B zugreifen, muss aber wegen der Sperre von Transaktion 2 warten.
- 4) Transaktion 2 will auf Tabelle A zugreifen, muss aber wegen der Sperre von Transaktion 1 warten.

Deadlocks - Beispiel

Abfrage-Fenster 1

```
-- Schritt 1
BEGIN TRANSACTION;
UPDATE Gehalt SET gehalt = 1000;
-- sperrt Tabelle Gehalt

-- Schritt 3
UPDATE Mitarbeiter SET stadt = 'Ulm';
-- wartet auf Tabelle Mitarbeiter
```

Abfrage-Fenster 2

```
-- Schritt 2
BEGIN TRANSACTION;
UPDATE Mitarbeiter SET stadt = 'München';
-- sperrt Tabelle Mitarbeiter

-- Schritt 4
UPDATE Gehalt SET gehalt = 2000;
-- wartet auf Tabelle Gehalt
-- DEADLOCK
```

Deadlock Priorität

- Über die Eigenschaft `DEADLOCK_PRIORITY` kann man Einfluss darauf nehmen, welche Transaktion abgebrochen wird.
- Mögliche Werte sind `LOW` | `NORMAL` | `HIGH` oder ein Integer-Wert im Bereich von -10 bis 10.
 - LOW => -5
 - NORMAL => 0
 - HIGH => 5
- Bei gleicher `DEADLOCK_PRIORITY` entscheidet wieder der SQL-Server über das Deadlock-Victim.